Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Арифметические операции с полиномами»

Выполнил:
студент группы 3824Б1ФИ2
Большаков В.В.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 7
3.3. Описание алгоритмов 12
4. Результаты тестирования 20
Заключение 22

Введение

Описание предметной области: предметом исследования являются алгебраические выражения, представленные в виде полиномов от трёх переменных (x,y и z), и операции над ними.

Описание задачи: реализация программы способной выполнять указанные арифметические действия (см. пункт «Постановка задачи») с полиномами трёх переменных с учетом ограничений на степень переменных.

Используемый инструмент для выполнения задачи: язык программирования C++, фреймворк **Google Test** для написания автоматического набора тестов, направленных на проверку корректности разработанной функциональности.

Сферы применения реализованной программы: учебная деятельность, связанная с математикой, физикой.

1. Постановка задачи

Постановка задачи: разработать программу на языке программирования C++, выполняющую арифметические операции с полиномами трех переменных (x, y и z): сложение, вычитание, умножение на константу, умножение двух полиномов. Считается, что полином составлен из мономов трех переменных с ограничением на степень каждой переменой от 0 до 9. Коэффициенты полинома - вещественные числа. Работоспособность программы проверяется с помощью автоматических тестов Google Test. Также необходимо реализовать пользовательское консольное приложение.

Условия и ограничения:

- Структура хранения полинома – односвязный список (реализуется самостоятельно)
- Степень полинома хранится в “свернутом виде”. Полином – множество мономов. Каждый моном представлен коэффициентом и степенью. Степень представлена как трехзначное число: число сотен – степень переменной “x”, число десятков – степень переменной “y”, число единиц - степень переменной “z” (т.е. степень – это число от 0 до 999).
- Сложение полиномов осуществляется алгоритмом слияния упорядоченных массивов.
- Вычитание полиномов допускается выполнять через сложение с умножением на константу (C = A - B = A + (-1) * B = A + B * (-1))
- При создании полинома, добавлении монома, сложении и умножении полиномов необходимо следить, чтобы подобные мономы были приведены. Также в итоговом полиноме не должно храниться мономов с нулевым коэффициентом.
- Если при умножении полиномов полученные степени переменных больше 9, выбрасывается исключение (в пользовательском приложении выводится сообщение об ошибке)
- Мономы в полиноме должны быть упорядочены по убыванию степеней.

2. Руководство пользователя

При включении программы пользователю выводится сообщение с просьбой выбрать режим начала работы программы (необходимо выбрать формат создания двух полиномов):

1. Polinom_1 пуст, Polinom_2 пуст
2. Polinom_1 не пуст, Polinom_2 пуст
3. Polinom_1 пуст, Polinom_2 не пуст
4. Polinom_1 не пуст, Polinom_2 не пуст.
0. Завершить работу программы.

Полином пуст: полином будет создан, пользователю не потребуется сразу же вводить мономы (пользователь сможет добавить мономы позже, но уже по одному за раз)

Полином не пуст: пользователь должен сразу ввести желаемое количество мономов. Формат ввода: 12:123,5:11,4:789. Мономы вводятся через запятую, пробелы и иные символы, кроме указанных, не допускаются. Слева от двоеточия стоит коэффициент, справа степень.

Если при выборе режима допущена ошибка (некорректное число: 9 (пример), или другие недопустимые символы), то ввод запрашивается до тех пор, пока пользователь не введет корректные числа.

После создания полиномов пользователю предлагается выбрать из списка возможную операцию:

1. Добавить моном в Polinom_1
2. Добавить моном в Polinom_2
3. Сложить Polinom_1 и Polinom_2 (результат операции будет выведен на экран)
4. Вычесть из Polinom_1 Polinom_2 (результат операции будет выведен на экран)
5. Вычесть из Polinom_2 Polinom_1 (результат операции будет выведен на экран)
6. Умножить Polinom_1 на константу (при этом будет изменен сам Polinom_1)
7. Умножить Polinom_2 на константу (при этом будет изменен сам Polinom_2)
8. Умножить Polinom_1 на Polinom_2 (результат операции будет выведен на экран)
9. Вывести Polinom_1
10. Вывести Polinom_2
0. Завершить работу программы

При добавлении мономов в полиномы пользователю нужно будет ввести коэффициент и степень. Ввод запрашивается, пока пользователь не введет корректные данные.

Если при умножении двух полиномов полученные степени больше 9, то выведется сообщение об ошибке. Пользователю будет предложено выбрать режим (1. Добавить моном в Polinom_1 и т.д.).

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из двух проектов: Lab№1.4 и Project1.

Проект Lab№1.4 содержит файл **test.cpp**, который содержит в себе все тесты Google Test. Данный проект предназначен в первую очередь для проверки работоспособности классов проекта Project1. * Включает вспомогательную функцию **check_correct**, которая сравнивает полученный результат выполнения теста с ожидаемым значением, позволяя определить работоспособность программы.

Перейдем к проекту **Project1**. В данном проекте два основных файла: **Source.cpp**, **Header.h**. В файле Header.h содержится реализация классов **F_List, Polinom**. Класс F_List является шаблонным и представляет собой структуру хранения “односвязный список”. Данный класс используется в реализации класса Polinom (подробнее о полях и методах данного класса и класса Polinom см. следующий пункт).

Класс Polinom предназначен для хранения и выполнения арифметических операций с полиномами (также присутствует глобальная функция для умножения константы на полином: 5 * polinom_1 (пример)).

Перейдем к файлу **Source.cpp**. Данный файл содержит реализацию пользовательского приложения. Он состоит из следующих функций:

- **main** – основная функция, запускающая работу программы. Позволяет пользователю выбрать формат создания двух полиномов. Также вызывает функцию **main_job**.
- **main_job** - данная функция позволяет пользователю выбрать арифметическую операцию между двумя полиномами (см. пункт “Руководство пользователя”).
- **correct_coeff -** запрашивает у пользователя ввод коэффициента для монома. Если пользователь вводит некорректные данные, то ввод запрашивается снова (выводит сообщение об ошибке).
- **correct_deg** – запрашивает у пользователя ввод степени монома. Если пользователь вводит некорректные данные, то ввод запрашивается снова (выводит сообщение об ошибке).
- **input_polinom** – функция, которая позволяет пользователю ввести несколько мономов (при создании полинома). Аналогично запрашивает повторный ввод, если пользователь ввел некорректные данные, и выводит сообщение о совершенный ошибке.
- **add_m** – вызывает функции correct_coeff и correct_deg. Сообщает правила ввода степени.
- **rules_add_monom** – выводит правила ввода полинома (перед использованием функции input_polinom).

3.2. Описание структур данных

Подключенные библиотеки: **iostream** (для операторов ввода и вывода), **vector** (для работы с классом std::vector), **string** (для преобразования числовых данных в строку и наоборот), **sstream** (объект класса std::stringstream используется для реализации оператора ввода класса Polinom), **cmath** (использование функции fabs в реализации оператора вывода класса Polinom). Определен макрос **SIZE_DEGREE** равный 3. Данный макрос используется для проверки количества цифр степеней мономов (должно быть не больше, чем данное значение)

Перейдем к описанию класса **F_List**.

Как было сказано в предыдущем пункте, данный класс является шаблонным и представляет собой реализацию “односвязного списка”.

Описание полей (спецификатор доступа: **private**):

- Содержит шаблонную структуру Node. Данная структура содержит несколько полей:
    - data – данные (шаблонного типа)
    - pNext – указатель на тип Node (т.е. содержит адрес следующего элемента Node)
    - ❖ Node(T data = T(), Node<T>* next = nullptr) – конструктор, который устанавливает значения соответствующих полей. Если не передан указатель, то поле pNext принимает значение nullptr. Если не переданы данные, то поле data принимает значение по умолчанию определенного типа (того же типа, что и объект класса F_List)
- Содержит класс Iterator. Данный класс содержит одно поле: указатель на объект типа Node (т.е. указатель на элемент класса F_List). Используется при реализации класса Polinom.
- head – указатель на тип Node (указывает на первый элемент списка)
- size – размер списка. Данное поле представлено типом size_t

Описание методов (спецификатор доступа: **public**):

F_List() – конструктор по умолчанию. Создает список размера 0, head получает значение nullptr.

F_List(const F_List& other) – конструктор копирования. Принимает ссылку на константный объект данного класса. Используется для копирования состояния переданного объекта.

Оператор присваивания: принимает ссылку на константный объект данного класса. Используется для копирования состояния переданного объекта. Возвращает ссылку на текущий объект (что позволяет применять данный оператор несколько раз подряд: list_1 = list_2…)

get_size(): возвращает размер списка (т.е. значение типа size_t)

push_back(): принимает ссылку на константу типа T (шаблонный тип). Используется для вставки данных, переданных в данный метод, в конец списка. Возвращает тип void.

T& operator [] (const size_t& index) – оператор индексации. Принимает ссылку на константный объект типа size_t (индекс элемента списка). Возвращает ссылку на тип данных T (что позволяет менять значения по индексу). Предназначен для получения и изменения значения по индексу.

const T& operator [] (const size_t& index) const – другая перегрузка оператора индексации. Позволяет только получать значения элемента списка по индексу (но не менять).

pop_front(): удаляет самый первый элемент списка. Возвращает тип void.

clear(): полностью очищает список (удаляет все элементы). Возвращает тип void. Использует метод pop_front.

~F_List() – деструктор. Очищает память, выделяемую при добавлении элементов в список (использует метод clear).

push_front(const T& value): принимает ссылку на константу типа T. Добавляет переданные данные в начало списка. Возвращает тип void.

empty(): возвращает значение типа bool. Если список пуст, то возвращает true. Иначе false.

insert(const T& value, const size_t& index): принимает ссылку на константу типа T (данные, которые нужно добавить в список), ссылку на константу типа size_t (индекс элемента, на место которого будет добавлен новый с переданными данными). Добавляет элемент с переданными данными на место элемента, индекс которого также передан в метод. Элемент, индекс которого передан в метод, будет передвинут на единицу вперед (его новый индекс: index+1). Возвращает тип void.

remove(const size_t& index): принимает ссылку на константу типа size_t. Данная константа – индекс элемента, который нужно удалить. Элемент, индекс которого равен index – 1 будет хранить указатель на элемент с индексом index + 1. Возвращает тип void.

Описание методов класса Iterator (спецификатор доступа: **public**):

Iterator(Node<T>* ptr = nullptr) – конструктор инициализации. Инициализирует единственное поле класса **current**. Если в конструктор не переданы данные, то поле current инициализируется значением nullptr.

operator*() – оператор разыменования. Возвращает ссылку на дынные, хранящиеся в узле current (т.е. можно не только получить данные, но и изменить их).

operator++() – префиксный инкремент. Перемещает указатель current на следующий узел. Возвращает ссылку на текущий объект данного класса.

operator++(int) – постфиксный инкремент. Создаёт копию текущего итератора, вызывает префиксный инкремент для текущего объекта и возвращает сохранённую копию.

operator !=(const Iterator& other) – оператор сравнения на неравенство. Принимает ссылку на константный объект данного класса. Сравнивает внутренние указатели (current) двух итераторов на неравенство. Возвращает тип bool. Является константным методом.

operator = =(const Iterator& other) – аналогичен оператору сравнения на неравенство, только сравнивает на равенство.

operator+=(size_t n) – оператор сложения с присвоением. В цикле n раз вызывает переход к следующему узлу. Возвращает ссылку на текущий объект данного класса.

Два метода класса F_List использующих класс Iterator (спецификатор доступа: **public**):

begin() – возвращает объект класса Iterator. Создает объект класса Iterator (в конструктор передает поле **head**). Является константным методом.

end() – аналогичен методу begin, но в конструктор класса Iterator передает nullptr.

Описание класса Polinom.

Данный класс используется для представления полиномов и работы с ними.

Описание полей (спецификатор доступа: **private**):

- структура **monom**:

Данная структура содержит два поля:

  - degree – представлено типом unsigned int (степень монома)
  - coeff - представлено типом double (коэффициент при мономе)

- Объект класса F_List, который хранит в себе данные типа monom.

Описание методов (спецификатор доступа: **private**):

QuickSort(std::vector<double>& coeff, std::vector<unsigned int>& degree, int left, int right) – принимает ссылку на объект типа std::vector, который хранит элементы типа double (вектор коэффициентов); ссылку на объект типа std::vector, который хранит элементы типа unsigned int (вектор степеней), левый индекс, правый индекс (оба типа int). Данный метод - реализация алгоритма быстрой сортировки. Упорядочивает элементы в векторах по возрастанию. Возвращает тип void.

push_front_monom(const double& coeff, const unsigned int& degree) – принимает ссылку на константный объект типа double, ссылку на константный объект типа unsigned int. Добавляет моном в начало списка (используется при реализации публичных методов, не содержит никаких проверок). Возвращает тип void.

count_digits(const unsigned int& value) – принимает ссылку на константный объект типа unsigned int. Это вспомогательный метод, который используется для подсчета количества цифр в передаваемой степени монома. Возвращает тип int.

Описание методов (спецификатор доступа: **public**):

Polinom(std::vector<double>& coeff, std::vector<unsigned int>& degree) – конструктор инициализации. Принимает ссылку на объект типа std::vector, который хранит элементы типа double; ссылку на объект типа std::vector, который хранит элементы типа unsigned int. Данные векторы – вектор коэффициентов и вектор степеней мономов. С помощью данного конструктора создается полином, содержащий переданные мономы. Пример:

Вектор коэффициентов: 5,6,78

Вектор степеней: 101,111,456

Итоговый полином: 78x4y5z6 + 6x1y1z1 + 5x1y0z1. Мономы в списке хранятся упорядоченно, т.к. в конструкторе применяется метод QuickSort. Также если есть мономы одинаковой степени, то они приводятся к одному моному. Мономы с нулевым коэффициентом не хранятся.

Polinom() – конструктор по умолчанию. Позволяет создать пустой полином (количество элементов в списке равно 0, т.к. данный конструктор вызывает конструктор по умолчанию класса F_List).

add_monom(const double& coeff, const unsigned int& degree) – принимает ссылку на константу типа double (коэффициент монома), принимает ссылку на константу типа unsigned int (степень монома). Добавляет моном в нужное место (т.е. гарантируется, что мономы в полиноме будут идти по убыванию степеней). Возвращает тип void.

Polinom(const Polinom& other) – конструктор копирования. Принимает ссылку на константный объект данного класса. Копирует состояние переданного объекта. Использует оператор присваивания класса F_List.

Polinom& operator = (const Polinom& other) – оператор присваивания. Принимаемое значение и реализация аналогичны конструктору копирования. Возвращает ссылку на объект данного класса, что позволяет использовать данный оператор несколько раз подряд (polinom_1 = polinom_2 = polinom_3 …).

Polinom operator + (const Polinom& other) – оператор сложения двух полиномов. Принимает ссылку на константный объект данного класса. Возвращает новый полином, содержащий в себе результат сложения двух полиномов (в итоговом полиноме приведены подобные слагаемые).

Polinom operator - (const Polinom& other) – оператор разности двух полиномов. Принимаемое значение - аналогично оператору сложения двух полиномов. Возвращает полином, содержащий в себе результат разности двух полиномов (в итоговом полиноме приведены подобные слагаемые).

Polinom operator* (const double& value) const – оператор умножения полинома на константу (константа находится справа от знака умножения). Является константным методом. Принимает ссылку на константный объект типа double. Возвращает полином, содержащий в себе результат умножения исходного полинома на константу.

Polinom operator* (const Polinom& other) – оператор умножения двух полиномов. Принимает ссылку на константный объект данного класса. Возвращает полином, содержащий в себе результат умножение двух полиномов (исходного и переданного). В итоговом полиноме также приведены подобные слагаемые.

Перегруженный оператор вывода: принимает ссылку на объект типа oustream, принимает ссылку на константный объект данного класса. Выводит данный полином в консоль. Формат вывода: 5x6y7z8 + x1y0z1 (цифры после переменных – это степени данных переменных, перед переменной x идут коэффициенты). Возвращает ссылку на объект ostream (что позволяет применять данный оператор несколько раз подряд: std::cout << polinom_1 << polinom_2 …)

Перегруженный оператор ввода: принимает ссылку на объект типа istream, ссылку на объект данного класса. Позволяет вводить мономы для данного полинома. Формат ввода: 12:123,4:456 (мономы идут через запятую, без пробелов. Слева от двоеточия указан коэффициент монома, справа степень монома). Возвращает ссылку на объект istream (что позволяет применять данный оператор несколько раз подряд: std::cin>>polinom_1>>polinom_2 и т.д.)

count_monom() – возвращает количество мономов в данном полиноме (т.е. размер списка). Возвращаемое значение имеет тип size_t.

Глобальная функция:

operator*(const double& value, const Polinom& polinom): принимает ссылку на константу типа double, принимает ссылку на константу класса Polinom. Позволяет умножать полином на константу, но константа находится слева от знака умножения. Возвращает новый полином, содержащий в себе результат умножения данного полинома на константу.

Операторы сравнения: принимают ссылки на константный объект данного класса. Сравнивают значения степени и коэффициента соответствующих мономов текущего полинома и переданного полинома. Возвращают тип bool (полиномы равны/не равны).

3.3. Описание алгоритмов

Описание алгоритмов класса **F_List**

Конструктор копирования: на вход подается ссылка на константный объект данного класса (const F_List& other). Если размер переданного объекта не ноль, то:

1. Полю size присваивается поле other.size
2. Создаем новый узел, содержащий копию данных первого узла переданного объекта (поле head содержит указатель на первый элемент)
3. В цикле while последовательно создаются новые узлы, каждый из которых хранит соответствующие данные переданного объекта. Каждый шаг цикла включает следующее:
   - Создание нового узла с копией данных.
   - Обновление указателей следующего узла (pNext) в исходном списке.
   - Продвижение указателя по переданному списку (изначально указывает на второй элемент).

Если размер переданного объекта ноль, то поля исходного списка получают соответствующие значение (size = 0, head = nullptr).
Сложность алгоритма: O(n), n – число элементов в переданном списке.

Оператор присваивания:

Если аргумент – не исходный список (проверка на самоприсваивание), то с помощью конструктора копирования создается временный список. Далее с помощью функции std::swap происходит обмен полей временного списка и исходного. Из-за использования конструктора копирования сложность алгоритма: O(n).

Метод **push_back**:

Если указатель на первый элемент списка равен nullptr (т.е. в списке нет элементов), то создается узел с переданными данными. Иначе цикл while проходится по списку. Когда поле узла pNext (указатель на следующий элемент) будет равно nullptr, тогда создается новый узел с переданными данными (т.е. в конце списка появляется новый элемент). Размер списка увеличивается на 1. Сложность алгоритма: O(n), где n – число элементов в исходном списке (если список пуст, то O(1)).

Оператор индексации:

Если размер списка равен 0, то выбрасывается исключение std::out_of_range. Также исключение выбрасывается, если переданный индекс больше или равен размеру списка.

Если проверка прошла успешно, то создается переменная count = 0. С помощью цикла while осуществляется проход по списку. Если count равен index, то возвращаются данные соответствующего элемента. Иначе count увеличивается на единицу, осуществляется переход к следующему элементу списка.

Сложность алгоритма: O(n), где n – число элементов в списке (в худшем случае)

Метод **pop_front**:

Создается временная переменная temp, которой присваивается адрес первого элемента (т.е. head). Полю head присваивается указатель на следующий элемент (pNext).

С помощью оператора delete освобождаются ресурсы, занятые первым элементом (delete temp). Размер уменьшается на 1. Сложность алгоритма O(1). Если список пуст, то осуществляется выход из метода.

Метод **clear**:

Пока число элементов в списке не равно 0, вызывается метод pop_front. Сложность алгоритма: O(n), где n – число элементов в списке.

Метод **push_front**:

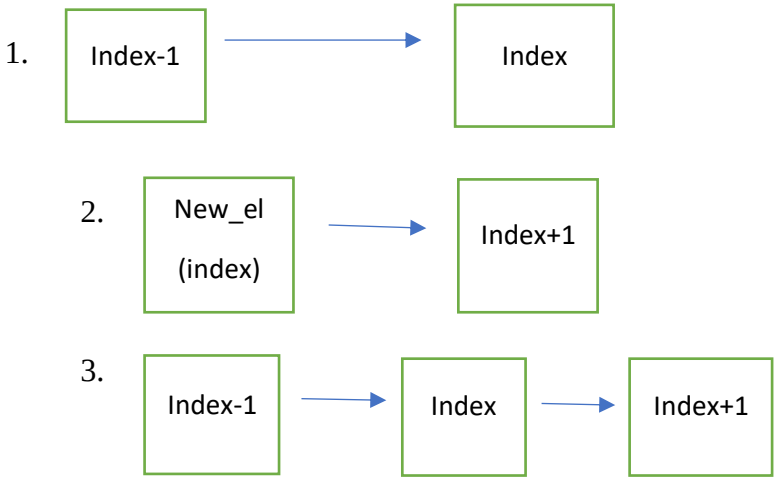
Создается новый узел, который хранит переданные данные и указывает на первый элемент изначального списка (первый элемент, до добавления данного элемента). Поле head указывает на добавленный элемент. Поле size увеличивается на 1.

Сложность алгоритма: O(1).

Метод **insert**:

Аналогично оператору индексации проверяется корректность переданного индекса. Если переданный индекс не корректен, то выбрасывается исключение.

Если проверка прошла успешно, то index проверяется на равенство нулю. Если он равен нулю, то вызывается метод push_front. Иначе цикл for проходит по списку до элемента с индексом index-1. После создается элемент, который указывает на элемент с индексом index (и хранит переданные данные). Элемент с индексом index-1 указывает на новый созданный элемент. Поле size увеличивается на 1.



Сложность алгоритма: O(n), где n – число элементов в списке (в худшем случае).

Метод **remove**:

Аналогично оператору индексации проверяется корректность переданного индекса. Если переданный индекс не корректен, то выбрасывается исключение. Создается указатель на тип Node, в котором будет хранится указатель на удаляемый элемент. Если удаляется первый элемент, то полю head присваивается указатель на следующий узел. Иначе с помощью цикла for находится элемент с индексом index – 1. Его полю pNext присваивается указатель на элемент с индексом index + 1. С помощью delete освобождаются ресурсы, выделенные под элемент с индексом index. Поле size

уменьшается на единицу.
Сложность алгоритма: O(n), где n - число элементов в списке (в худшем случае)

В классе Iterator все методы имеют сложность O(1), кроме operator +=. «В цикле n раз вызывает переход к следующему узлу» (останавливается, когда поле pNext становится равным nullptr). Поэтому его сложность: O(n).

Описание алгоритмов класса **Polinom**:

Приватный метод **QuickSort**:

1. Переменные i и j указывают соответственно на левый и правый края диапазона сортируемого вектора (i = left, j = right). После вычисляется значение опорного элемента (mid = degree[(i+j)/2]).
2. Цикл do…while. Если элемент слева (degree[i] – степень i- го монома) меньше опорного, индекс i увеличивается (внутренний цикл while). Если элемент справа (degree[j]) больше опорного, индекс j уменьшается (внутренний цикл while). После двух циклов while проверяется условие i<=j. Если оно выполняется, то с помощью функции std::swap меняем местами элементы по индексами i и j обоих векторов (векторов degree, coeff). Индексы смещаются далее. Цикл do…while работает пока i<j. Таким образом:
   - все элементы меньше опорного оказываются слева от него,
   - все элементы больше опорного оказываются справа
   - элементы, равные опорному, могут оказаться либо слева, либо справа от опорного
   - (если условие i<=j нарушается, то значит элементы отсортированы относительно опорного элемента).
3. Далее происходит рекурсивный вызов функции для отрезка [i, right] (если i<right), а после для отрезка [left, j] (если j>left).
   Сложность алгоритма: в среднем $O(n \log_2(n))$, в худшем случае $O(n^2)$.

   Разбор среднего случая: шаг сортировки относительно опорного элемента выполняется за O(n) (т.е. мы проходимся в итоге по всем элементам вектора). Если вектор рекурсивно делится каждый раз пополам, то глубина рекурсии составит $\log_2(n)$ (n – размер вектора. На k шаге длина подвектора составит $n/2^k$. Хотим узнать на каком шаге длина составит 1. Это произойдет при $k = \log_2(n)$).

   Худший случай: выбран плохой опорный элемент. Тогда каждое деление приведет к неравномерному разделению. В таком случае глубина рекурсии станет линейной.

Конструктор инициализации:

1. Проверка совпадения размеров переданных векторов. В случае не совпадения выбрасывается исключение **std::invalid_argument**.
2. Цикл for проходится по всем элементам вектора degree. С помощью метода **count_digits** происходит подсчет количества цифр в элементах вектора. Если число цифр в каком-либо элементе больше чем 3, то выбрасывается исключение **std::invalid _argument**.

3. Создаются два вектора (co – вектор коэффициентов, deg – вектор степеней), в которые с помощью цикла for помещаются элементы переданных векторов (если элемент вектора coeff равен 0, то данный элемент и элемент вектора degree под тем же индексом не добавляются).
4. Если размер вектора deg больше 1, то для “co” и “deg” вызывается метод QuickSort.
5. После с помощью цикла for и вложенного в него цикла while происходит приведение подобных слагаемых. Если в векторе deg присутствуют одинаковые элементы, то элементы вектора “co” под соответствующими индексами складываются. Если нет, то ничего не происходит. После цикла while вызывается метод push_front_monom, который добавляет элементы в поле list.

   Сложность алгоритма:
   - проверка элементов вектора degree: O(n)
   - заполнение векторов “co” и “deg”: O(n)
   - метод QuickSort: $O(n\log_2(n))$
   - приведение подобных слагаемых в отсортированных векторах и добавление элементов в поле list: O(n)

   Итоговая сложность: $O(n + n\log_2(n))$

В методах: add_monom, оператор сложения двух полиномов, оператор умножения полинома на константу, оператор умножения двух полиномов, оператор вывода полинома - применяются итераторы (объекта класса Iterator), поэтому доступ к мономам (соответственно к их данным) осуществляется за O(1).

Метод **add_monom**:

1. Проверяется, соответствует ли передаваемая степень заданному формату (три цифры). Если степень записана неверно (более трех цифр), выбрасывается исключение **std::invalid_argument.**
2. Моном будет добавлен, если переданный коэффициент не равен 0.
3. Если размер списка равен 0, то методом push_front моном добавляется в список.
4. Если размер списка не равен 0, то создается моном с полями равными переданным аргументам. После идет цикл for, проходящий до конца списка.
   - Если переданная степень больше степени i-го элемента списка, то с помощью метода insert моном добавляется на место i-го элемента (цикл завершается).
   - Если переданная степень равна степени i-го элемента списка, то коэффициент i-го элемента складывается с переданным коэффициентом. Полученный коэффициент заменяет старый коэффициент i-го элемента (цикл завершается). Если полученный коэффициент равен 0, то вызывается метод remove, удаляющий i -ый элемент списка.
   - Если список пройден, а моном не добавлен в список, то вызывается метод push_back, добавляющий моном в конец списка.

   Сложность алгоритма:
   - Цикл for: O(n)
   - Метод insert: O(n).
   - Метод push_back: O(n)
   - Метод remove: O(n)

Итоговая сложность: O(n).

Оператор сложения двух полиномов (алгоритм слияния):

1. Сначала создаются объекты класса Iterator: it1 – указывает на первый элемент списка текущего полинома (первого полинома), it2 – на первый элемент списка переданного полинома (второго полинома). Создается итоговый полином result (пустой на данный момент), временный полином temp (также пустой).
2. Далее идет цикл while:
    - Если степени совпадают, коэффициенты соответствующих мономов суммируются, и, если полученный коэффициент не равен нулю, то результат добавляется в temp (методом push_front_monom).
    - Если степень текущего монома первого полинома больше степени монома второго полинома, то моном первого полинома добавляется в temp.
    - Иначе в полином добавляется моном из второго полинома.
3. Если один из полиномов остался не пройденным до конца, то с помощью цикла while и метода push_front_monom в temp добавляются оставшиеся мономы.
4. Далее с помощью ещё одного цикла for мономы из temp добавляются в result (это необходимо для упорядоченного хранения мономов в списке (по убыванию))

Сложность алгоритма: пусть m – размер первого полинома, n – размер второго полинома. Итоговая сложность O(m+n)

Оператор умножения полинома на константу:

1. Если размер текущего полинома равен нулю, то данный полином возвращается без изменений.
2. Создается итоговый полином pol (пустой на данный момент), временный полином pol_temp (также пустой)
3. Если переданный коэффициент не равен 0, то цикл for проходит по элементам текущего полинома начиная с первого. Коэффициенты мономов умножаются на переданное значение. Методом push_front_monom новые мономы добавляются в pol_temp (степени мономов остались без изменений). После с помощью ещё одного цикла for мономы из pol_temp добавляются в итоговый полином (методом push_front_monom).
4. Если переданный коэффициент равен 0, то возвращается пустой полином. Сложность алгоритма: O(n)

Оператор умножения полинома на полином:

1. Если один из полиномов пустой, то возвращается пустой полином.
2. Иначе создаются два вспомогательных вектора: result_degr, result_coeff.
3. Далее идет цикл for проходящий по мономам первого полинома.
    - Данный цикл содержит вложенный цикл for, проходящий по мономам второго полинома (переданного в качестве аргумента данного оператора). С помощью данных циклов происходит вычисление новых коэффициентов и новых степеней итогового полинома.
    - Если новая степень содержит больше трёх цифр, то выбрасывается исключение.

- Первый новый коэффициент (new_coeff) и новая степень (new_degr) сразу добавляются в соответствующие векторы.
- После идет ещё один цикл for, проходящий по элементам вектора result_degr. Если new_degr равна одному из элементов вектора result_degr, то элемент вектора result_coeff под текущим индексом (пусть это элемент A) заменяется на новый, равный сумме new_coeff и A. Как только это произошло, данный цикл прекращает работу.
- Если вектор result_degr был полностью пройден, то вычисленные коэффициент и степень добавляются в соответствующие векторы.

4. После создается итоговый полином (для создания используется соответствующий конструктор и сформированные векторы result_degr, result_coeff)
   Сложность алгоритма: пусть m – размер текущего полинома, n – размер второго полинома, r – число элементов в векторе result_degr.
   - Сложность двух первых циклов for: O(m*n)
   - Сложность третьего цикла for: в худшем случае все степени будут уникальны (т.е. количество степеней будет m*n). Поэтому его сложность O((m*n))
   - Сложность использования конструктора при создании итогового полинома: $O(r + r\log_2(r))$.
     Итоговая сложность: $O((m*n)^2)$.

Оператор вывода полинома:

1. Создается массив символов, содержащий переменные: x, y, z
2. Цикл for проходит по элементам поля list (i – индекс элемента списка):
   - Если i не ноль, то в зависимости от знака коэффициента i-го монома выводится “+” или “-“
   - Если i равен нулю и коэффициент первого монома отрицательный, то выводится “-“
   - Если коэффициент i-го монома не равен 1/-1 то выводится модуль коэффициента
   - Если степень не равна 0 (степень мономов представлена типом unsigned int):
     - Степень преобразуется в строку (функция std::to_string)
     - Строка степени дополняется ведущими нулями до трех знаков (цикл while)
     - Для каждой цифры степени выводится соответствующая переменная и ее показатель степени (внутренний цикл for)

Сложность алгоритма: O(n), n – число элементов в полиноме (оба цикла for вместе работают за O(n)).

Оператор ввода полинома:

1. Создаются два вектора: coeff – вектор коэффициентов, deg – вектор степеней.
2. С помощью функции std::getline пользователь вводит полином в строку str
3. Создается объект inp класса std::stringstream. В него передается строка str.
4. Создается строка token и переменная number_monom равная нулю (отвечает за номер монома)

5. С помощью цикла while и функции std::getline разбиваем строку на элементы (ввод мономов происходит через запятую, поэтому разбиение происходит по символу “,”):
    - С помощью метода find класса std::string ищем позицию символа “ : “. Если позиция не найдена, то выбрасывается исключение.
    - Создаются две строки: co – подстрока, содержащая символы до двоеточия, degr – подстрока, содержащая символы после двоеточия.
    - Две проверки: размер “co” равен 0 или размер “degr” равен 0. В обоих случаях выбрасывается исключение **std::runtime_error**.
    - Переменная number_monom увеличивается на 1.
    - Цикл for проходит по строке “co”. В данном цикле проводятся проверки на корректный ввод коэффициента (например: количество точек не больше 1 и т.д.). В случае провала проверки выбрасывается исключение **std::runtime_error.** Также выводится сообщение об ошибке с номером монома, в котором она была допущена
    - Второй цикл for проходится по строке degr. Если в строке присутствуют не только символы цифр, то выбрасывается исключение аналогично предыдущему циклу for.
    - Строки “co” и “degr” преобразуются в тип double и тип unsigned int соответственно. После чего добавляются в векторы coeff и deg.
6. Далее идет блок try…catch, в котором происходит попытка создания итогового полинома. Если возникло исключение, то оно вновь бросается в блоке catch. Иначе созданный полином присваивается переданному (с помощью оператора присваивания)

Сложность алгоритма: пусть l – длина всей введенной строки. Все циклы вместе проходят по всей строке. Пусть число мономов равно n. Сложность работы конструктора: $O(n+n\log_2(n))$. Итоговая сложность: $O(l + n + n\log_2(n))$

Оператор сравнения на равенство:

1. Проверяются размеры полиномов (если число мономов не совпадает, то возвращается false)
2. В цикле for проверяется совпадение коэффициентов и степеней соответствующих мономов (мономы хранятся упорядочено). Если есть хотя бы одно несовпадение, то возвращается false.
3. Если проверки прошли успешно, то возвращается true.
   Сложность алгоритма: O(n).

Оператор сравнения на неравенство: вызывает оператор сравнения на равенство и инвертирует его результат.
Сложность алгоритма: O(n).

Метод **count_digits**:

На вход подается число типа unsigned int, для которого нужно посчитать количество входящих в него цифр (ссылка на константу типа unsigned int). Заводится счётчик count равный нулю, переменная val, которая инициализируется переданным значением. После

идет цикл while, в котором переменная val делится без остатка на 10, после чего count увеличивается на 1. Цикл работает пока val не станет равной нулю. Count возвращается в качестве результата. Сложность алгоритма: O(n), где n – количество цифр числа (но данный метод применяется только к степеням мономов, которые содержат не более 3-х цифр. Поэтому сложность алгоритма: O(1))

Метод **push_front_monom:**

В методе создается моном, полям которого присваиваются переданные аргументы. После с помощью метода push_front данный моном добавляется в поле list. Сложность алгоритма: O(1)

Конструктор по умолчанию: вызывается у поля list конструктор по умолчанию. Сложность алгоритма: O(1)

Конструктор копирования:

Аргумент конструктора – ссылка на константный объект данного класса. С помощью оператора присваивания полю list текущего объекта присваивается поле list аргумента. Сложность алгоритма: O(n).

Оператор присваивания: аналогично конструктору копирования. Возвращается ссылка на созданный объект.

Оператор разности двух полиномов:

Создается новый полином, равный переданному аргументу, умноженному на (-1). После к текущему полиному и только что созданному применяется оператор сложения двух полиномов. Итоговый полином возвращается в качестве результата.

Сложность алгоритма:

- Умножение на константу: O(n)
- Конструктор копирования: O(n)
- Оператор сложения двух полиномов: O(m+n), m – размер текущего полинома, n – размер созданного полинома (равен размеру аргумента).

Итоговая сложность: O(m+n)

Метод **count_monom**:

Возвращает результат метода get_size поля list. Сложность алгоритма: O(1). Является константным методом.

Глобальная функция умножения константы на полином:

Создается новый полином, в который записывается результат умножения аргумента (полином) на константу. Созданный полином возвращается в качестве результата.

Сложность алгоритма: O(n).

Результаты тестирования

Тесты Google Test для класса F_List (единая группа тестов test_forward_list):

- В тесте test0 проверяется корректность работы следующих методов: push_front, get_size, оператор индексирования, clear, empty, push_back, pop_front. Также в данном тесте проверялась корректность выброса исключений для оператора индексирования.
- В тестах test_operator_equal, test_operator_equal1, test_operator_equal2 проверяется корректность работы оператора присваивания (присваивание одному полиному другого, несколько присваиваний подряд)
- В тесте test_insert проверяется корректность работы метода insert, также проверяется корректность выброса исключений при передаче в данный метод некорректных данных (некорректный индекс для вставки).
- В тесте test_remove проверяется корректность работы метода remove, также проверяется корректность выброса исключений при передаче некорректного индекса.
- В тестах test_iterator – test_iterator2 проверяется корректность работы методов класса Iterator (разыменования, постфиксный инкремент, инфиксный инкремент и др.). Также проверяется корректность выброса исключений (оператор разыменования), корректность работы методов begin(), end().

Функция **check_correct**:

- Описание параметров: принимает ссылку на объект класса Polinom, ссылку на объект класса std::string.
- Описание реализуемого функционала: создается объект класса ostringstream, в который с помощью перегруженного оператора вывода класса Polinom помещается переданный полином в виде строки. Данная строка сравнивается на равенство с переданной строкой (т.е. сравниваются полином, который получился в результате операции, и полином, который должен был получится). Сравнение осуществляется с помощью макроса EXPECT_EQ.

Тесты Google Test для класса Polinom:

Группа тестов test_polinom0:
В тестах test_constructor – test_constructor3 проверяется:

- Корректность работы конструктора инициализации (создание полинома при передаче вектора коэффициентов и вектора степеней; приведение подобных мономов, если такие были переданы).
- Корректность выброса исключений при передаче в конструктор некорректных данных (количество коэффициентов не совпадает с количеством степеней; какая-либо из степеней имеет больше 3-х цифр)
- Корректность работы конструктора по умолчанию (создание пустого полинома)

В тестах test_add_monom – test_add_monom5 проверяется:

- Корректность работы метода add_monom (добавление монома в корректное место в полиноме, приведение подобных мономов, удаление монома при появлении

- нулевого коэффициента)

В тесте test_constructor_copy проверяется корректность работы конструктора копирования, в тесте test_operator_equal проверяется корректность работы оператора присваивания.

Группа тестов test_polinom1 (арифметические операции с полиномами):
В тестах test_multip_const – test_multip_const2 проверяется корректность работы оператора умножения полинома на константу (изменение коэффициентов мономов, умножение пустого полинома на константу, умножение полинома на 0)

В тестах test_operator_sum – test_operator_sum3 проверяется корректность работы оператора сложения двух полиномов (корректность сложения полиномов, приведение подобных мономов, сложение пустого и непустого полиномов)

В тестах test_operator_diff проверяется:
- Корректность работы оператора разности двух полиномов (корректность разности полиномов, приведение подобных)
- Корректность глобальной функции умножения константы на полином (возможность производить вычитание через сложение с умножением на константу: Polinom = Polinom_1 – Polinom_2 = Polinom_1 + (-1) * Polinom_2)

В тестах test_operator_multip_polinom – test_operator_multip_polinom5 проверяется:
- Корректность работы оператора умножения двух полиномов (корректность умножения полиномов, приведение подобных мономов, умножение пустого полинома на непустой)
- Корректность выброса исключений (если степень какого-либо монома превысит 3 знака (т.е. какая-либо из переменных имеет степень больше 9))

Группа тестов test_polinom2 (операторы ввода и вывода):

В тесте test_operator_output проверяется корректность работы оператора вывода полинома (применение нескольких операторов подряд)

В тестах test_input – test_input5 проверяется:
- Корректность работы оператора ввода
- Применение нескольких операторов подряд
- Выброс исключений при вводе некорректных данных (ввод некорректных символов (пробелы и т.д.), знак “-“ в некорректном месте и т.д.)

Группа тестов test_polinom3 (операторы сравнения):
В данной группе тестов (тесты test_operator_comparison, test_operator_comparison1) проверяется корректность работы операторов сравнения полиномов.

Данные тесты направлены на проверку корректности работы методов класса F_List и Polinom. Необходимой задачей было написание и прохождение всех тестов. Благодаря этому можно сказать, что методы классов F_List и Polinom работают корректно.

5. Заключение

В ходе работы достигнуты следующие результаты:

- Реализован класс F_List (одна из возможных реализаций «односвязного списка»)
- Реализован класс Polinom, позволяющий совершать следующие арифметические операции с полиномами трех переменных (с ограничением на степень каждой переменной: 0 – 9): сложение, вычитание, умножение на константу, умножение двух полиномов.
- Написаны тесты Google Test, направленные на проверку корректности реализации классов F_List, Polinom, работу методов данных классов при некорректных данных.
- Реализовано пользовательское консольное приложение, предоставляющее пользователю интерфейс для ввода полиномов, выбора необходимых операций и отображения результатов вычислений